Department of Information and Communication Technology

Faculty of Technology

University of Ruhuna

Database Management Systems Practicum

ICT 1222

Assignment 02 – Mini Project

Group Number 15

Submitted to: Mr.P.H.P. Nuwan Laksiri

Submitted by:

TG/2023/1700-A.L.B.Abilasha

TG/2023/1711-B.P.Kothalawala

TG/2023/1716-D.G.Induwara

TG/2023/1743-S.K.A.C.J.Samarasinghe

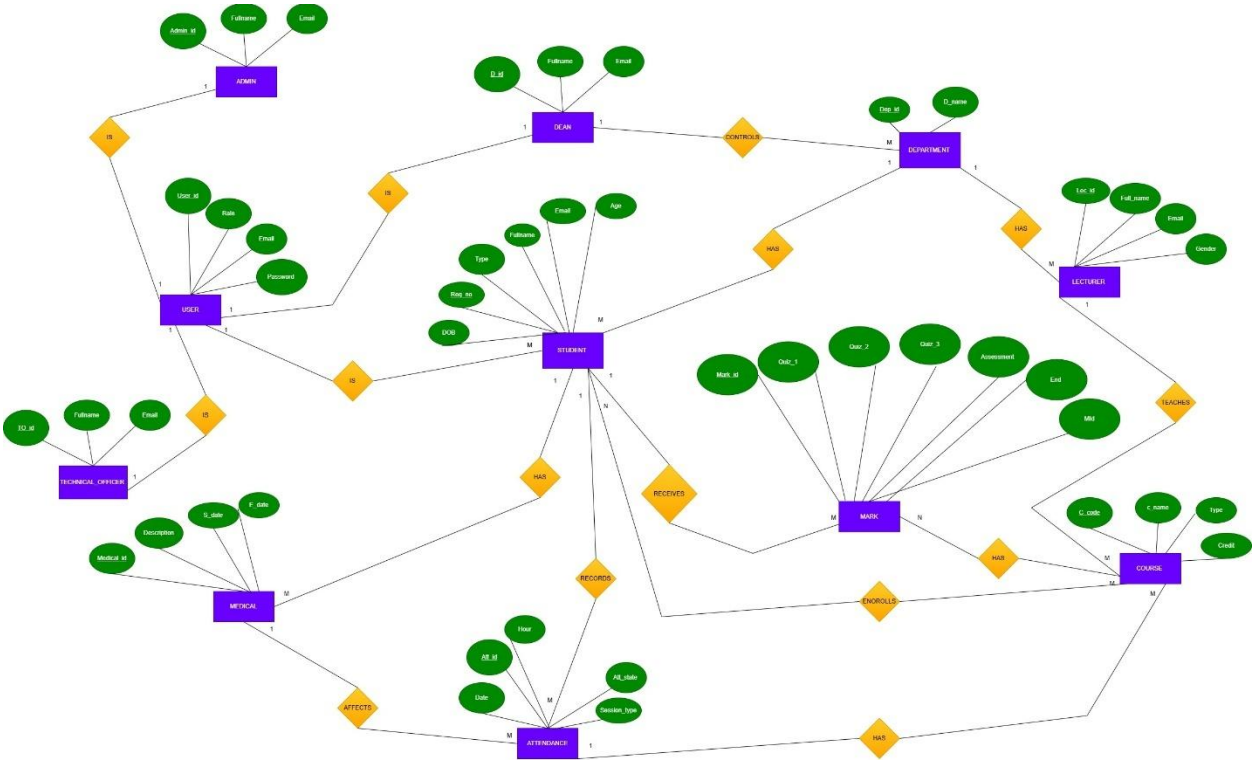
Brief Introduction about the Problem/Group Project

The Faculty of Technology at the University of Ruhuna manages a large volume of student and staff information, including attendance, marks, and course records.
Handling these processes manually increases the likelihood of errors, redundancy, and inefficiency.
Therefore, we developed an automated Faculty Management Information System (FacultyDB) to simplify data handling, ensure accuracy, and improve accessibility and security across departments.

Brief Introduction to the Solution

FacultyDB is a relational database system designed to automate student and course management activities within the Faculty of Technology.
The database supports data storage, retrieval, and security for users such as students, lecturers, the dean, technical officers, and administrators.
It eliminates redundancy through normalization and enforces data integrity using foreign keys and constraints.
The system also provides data-driven insights through queries, views, and stored procedures.

Proposed ER/EER Diagram



Proposed Relational Mapping Diagram

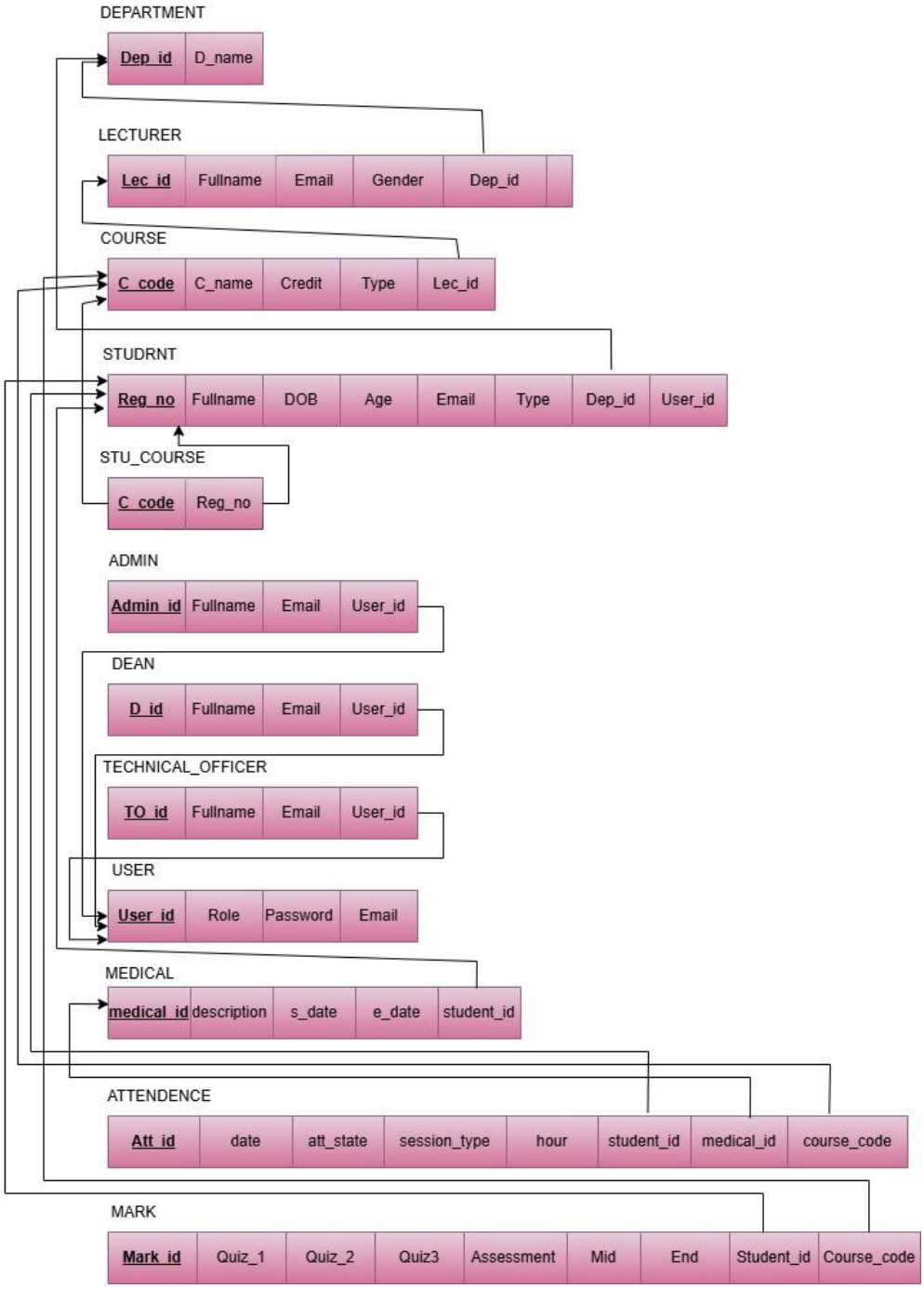


Table Structure of the Solution

CREATE Department Table

```
CREATE TABLE Department (
   Dep_id VARCHAR(10) PRIMARY KEY,
   D_name VARCHAR(100) NOT NULL
);
```

```
mysql> DESC Department;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| Dep_id  | varchar(10)  | NO   | PRI | NULL    |       |
| D_name  | varchar(100) | NO   |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
```

CREATE Lecturer Table

```
CREATE TABLE Lecturer (
   Lec_id VARCHAR(10) PRIMARY KEY,
   Fullname VARCHAR(100) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Gender ENUM('Male', 'Female', 'Other'),
   Dep_id VARCHAR(10),
   FOREIGN KEY (Dep_id) REFERENCES Department(Dep_id)
      ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
mysql> desc Lecturer;
+----------+-------------------------------+------+-----+---------+-------+
| Field    | Type                          | Null | Key | Default | Extra |
+----------+-------------------------------+------+-----+---------+-------+
| Lec_id   | varchar(10)                   | NO   | PRI | NULL    |       |
| Fullname | varchar(100)                  | NO   |     | NULL    |       |
| Email    | varchar(100)                  | YES  | UNI | NULL    |       |
| Gender   | enum('Male','Female','Other') | YES  |     | NULL    |       |
| Dep_id   | varchar(10)                   | YES  | MUL | NULL    |       |
+----------+-------------------------------+------+-----+---------+-------+
```

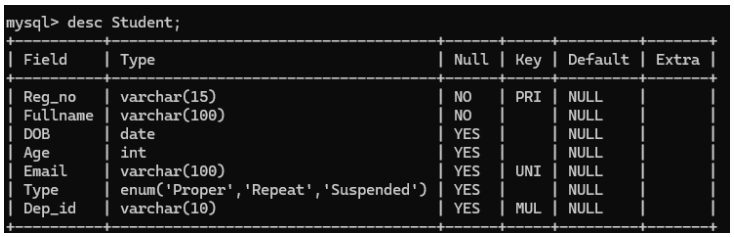
CREATE Course Table

```
CREATE TABLE Course (
   C_code VARCHAR(10) PRIMARY KEY,
   C_name VARCHAR(100) NOT NULL,
   Credit INT,
   Type ENUM('Theory', 'Practical'),
   Lec_id VARCHAR(10),
   FOREIGN KEY (Lec_id) REFERENCES Lecturer(Lec_id)
      ON DELETE SET NULL ON UPDATE CASCADE
);
```

```
mysql> desc Course;
+---------+-----------------------------+------+-----+---------+-------+
| Field   | Type                        | Null | Key | Default | Extra |
+---------+-----------------------------+------+-----+---------+-------+
| C_code  | varchar(10)                 | NO   | PRI | NULL    |       |
| C_name  | varchar(100)                | NO   |     | NULL    |       |
| Credit  | int                         | YES  |     | NULL    |       |
| Type    | enum('Theory','Practical') | YES  |     | NULL    |       |
| Lec_id  | varchar(10)                 | YES  | MUL | NULL    |       |
+---------+-----------------------------+------+-----+---------+-------+
```

CREATE Student Table

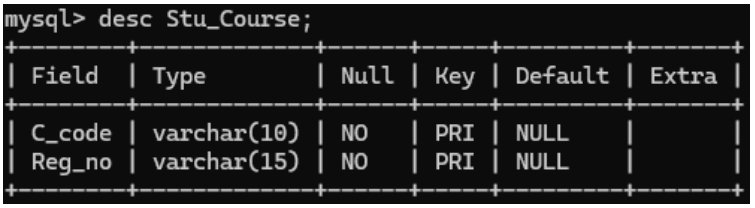
```
CREATE TABLE Student (
   Reg_no VARCHAR(15) PRIMARY KEY,
   Fullname VARCHAR(100) NOT NULL,
   DOB DATE,
   Age INT,
   Email VARCHAR(100) UNIQUE,
   Type ENUM('Proper', 'Repeat', 'Suspended'),
   Dep_id VARCHAR(10),
   FOREIGN KEY (Dep_id) REFERENCES Department(Dep_id)
      ON DELETE SET NULL ON UPDATE CASCADE
);
```



```
mysql> desc Student;
+----------+-----------------------------------+------+-----+---------+-------+
| Field    | Type                              | Null | Key | Default | Extra |
+----------+-----------------------------------+------+-----+---------+-------+
| Reg_no   | varchar(15)                       | NO   | PRI | NULL    |       |
| Fullname | varchar(100)                      | NO   |     | NULL    |       |
| DOB      | date                              | YES  |     | NULL    |       |
| Age      | int                               | YES  |     | NULL    |       |
| Email    | varchar(100)                      | YES  | UNI | NULL    |       |
| Type     | enum('Proper','Repeat','Suspended') | YES  |     | NULL    |       |
| Dep_id   | varchar(10)                       | YES  | MUL | NULL    |       |
+----------+-----------------------------------+------+-----+---------+-------+
```

CREATE Student-Course Relationship

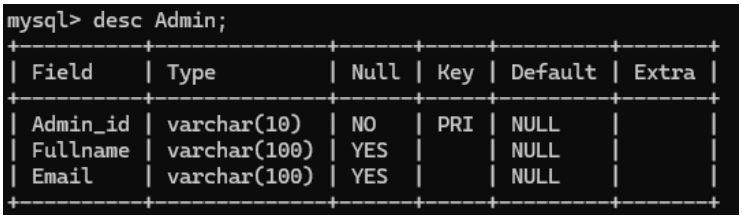
```
CREATE TABLE Stu_Course (
   C_code VARCHAR(10),
   Reg_no VARCHAR(15),
   PRIMARY KEY (C_code, Reg_no),
   FOREIGN KEY (C_code) REFERENCES Course(C_code)
      ON DELETE CASCADE ON UPDATE CASCADE,
   FOREIGN KEY (Reg_no) REFERENCES Student(Reg_no)
      ON DELETE CASCADE ON UPDATE CASCADE
);
```



```
mysql> desc Stu_Course;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| C_code | varchar(10) | NO   | PRI | NULL    |       |
| Reg_no | varchar(15) | NO   | PRI | NULL    |       |
+--------+-------------+------+-----+---------+-------+
```

CREATE Admin Table

```
CREATE TABLE Admin (
   Admin_id VARCHAR(10) PRIMARY KEY,
   Fullname VARCHAR(100),
   Email VARCHAR(100)
);
```



```
mysql> desc Admin;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| Admin_id | varchar(10) | NO   | PRI | NULL    |       |
| Fullname | varchar(100) | YES  |     | NULL    |       |
| Email    | varchar(100) | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
```

CREATE Dean Table

```
CREATE TABLE Dean (
   D_id VARCHAR(10) PRIMARY KEY,
   Fullname VARCHAR(100),
   Email VARCHAR(100)
);
```

```
mysql> desc Dean;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| D_id     | varchar(10)  | NO   | PRI | NULL    |       |
| Fullname | varchar(100) | YES  |     | NULL    |       |
| Email    | varchar(100) | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

CREATE Technical Officer Table

```
CREATE TABLE Technical_Officer (
   TO_id VARCHAR(10) PRIMARY KEY,
   Fullname VARCHAR(100),
   Email VARCHAR(100)
);
```

```
mysql> desc Technical_Officer;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| TO_id    | varchar(10)  | NO   | PRI | NULL    |       |
| Fullname | varchar(100) | YES  |     | NULL    |       |
| Email    | varchar(100) | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

CREATE User Table

```
CREATE TABLE User (
   User_id VARCHAR(10) PRIMARY KEY,
   Role ENUM('Admin', 'Dean', 'Lecturer',
'Technical_Officer', 'Student'),
   Password VARCHAR(100) NOT NULL,
   Email VARCHAR(100) UNIQUE
);
```

```
mysql> desc User;
+----------+-----------------------------------------------------------------------+------+-----+---------+-------+
| Field    | Type                                                                  | Null | Key | Default | Extra |
+----------+-----------------------------------------------------------------------+------+-----+---------+-------+
| User_id  | varchar(10)                                                           | NO   | PRI | NULL    |       |
| Role     | enum('Admin','Dean','Lecturer','Technical_Officer','Student')         | YES  |     | NULL    |       |
| Password | varchar(100)                                                          | NO   |     | NULL    |       |
| Email    | varchar(100)                                                          | YES  | UNI | NULL    |       |
+----------+-----------------------------------------------------------------------+------+-----+---------+-------+
```

CREATE medical Table

```
CREATE TABLE Medical(
   medical_id CHAR(10) PRIMARY KEY,
   description  VARCHAR(50),
   s_date DATE,
   e_date DATE,
   student_id VARCHAR(6),
   FOREIGN KEY (student_id) REFERENCES Student(Reg_no)
         ON DELETE CASCADE ON UPDATE CASCADE

   );
```

```
mysql> desc Medical;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| medical_id  | char(10)    | NO   | PRI | NULL    |       |
| description | varchar(50) | YES  |     | NULL    |       |
| s_date      | date        | YES  |     | NULL    |       |
| e_date      | date        | YES  |     | NULL    |       |
| student_id  | varchar(6)  | YES  | MUL | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

CREATE atttendence Table

```
CREATE TABLE Attendence(
   att_id VARCHAR(5) PRIMARY KEY,
   date DATE,
   att_state VARCHAR(20),
   session_type VARCHAR(25),
   hour INT,
   student_id VARCHAR(6),
   medical_id CHAR(10),
   course_code CHAR(8),
   FOREIGN KEY (student_id) REFERENCES Student(Reg_no)
        ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (course_code) REFERENCES Course(C_code)
        ON DELETE CASCADE ON UPDATE CASCADE,
   FOREIGN KEY (medical_id) REFERENCES Medical(medical_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
mysql> desc Attendence;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| att_id       | varchar(5)  | NO   | PRI | NULL    |       |
| date         | date        | YES  |     | NULL    |       |
| att_state    | varchar(20) | YES  |     | NULL    |       |
| session_type | varchar(25) | YES  |     | NULL    |       |
| hour         | int         | YES  |     | NULL    |       |
| student_id   | varchar(6)  | YES  | MUL | NULL    |       |
| medical_id   | char(10)    | YES  | MUL | NULL    |       |
| course_code  | char(8)     | YES  | MUL | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
```

CREATE Mark Table

```
CREATE TABLE Mark(
   mark_id CHAR(10) PRIMARY KEY,  quiz_1 INT,quiz_2 INT,
   quiz_3 INT, assesment INT, mid INT,end INT,
   student_id VARCHAR(6),
   course_code char(8),
   CONSTRAINT chk_marks CHECK (
  quiz_1  BETWEEN 0 AND 100 AND
  quiz_2  BETWEEN 0 AND 100 AND
   quiz_3  BETWEEN 0 AND 100 AND
   assesment  BETWEEN 0 AND 100 AND
    mid  BETWEEN 0 AND 100 AND
    end  BETWEEN 0 AND 100 ),
   FOREIGN KEY (course_code) REFERENCES Course(C_code)
        ON DELETE CASCADE ON UPDATE CASCADE,
   FOREIGN KEY (student_id) REFERENCES Student(Reg_no)
        ON DELETE CASCADE ON UPDATE CASCADE

);
```

```
mysql> desc Mark;
+-------------+------------+------+-----+---------+-------+
| Field       | Type       | Null | Key | Default | Extra |
+-------------+------------+------+-----+---------+-------+
| mark_id     | char(10)   | NO   | PRI | NULL    |       |
| quiz_1      | int        | YES  |     | NULL    |       |
| quiz_2      | int        | YES  |     | NULL    |       |
| quiz_3      | int        | YES  |     | NULL    |       |
| assesment   | int        | YES  |     | NULL    |       |
| mid         | int        | YES  |     | NULL    |       |
| end         | int        | YES  |     | NULL    |       |
| student_id  | varchar(6) | YES  | MUL | NULL    |       |
| course_code | char(8)    | YES  | MUL | NULL    |       |
+-------------+------------+------+-----+---------+-------+
```

Architecture of the Solution

The architecture follows a simple three-layer design:
1. Data Layer – MySQL database for storing faculty data.
2. Logic Layer – SQL scripts, stored procedures, and views for data operations.
3. Presentation Layer – Potential future integration with a web interface or backend API.

Tools and Technologies Used

- Draw.io
  Used to create the ER-diagram and Relational schema.
- MYSQL server, Visual studio code, Notepad++
  To create and manage database efficiency.
  To create table structures and data insertion.
- GitHub and GitHub desktop
  To manage, collaborate on database and version control.
- Microsoft word
  To create Final report and SRS report.

Security Measures to Protect the Database

- Admin: All privileges with GRANT OPTION.
- Dean: All privileges without GRANT.
- Lecturer: All privileges without GRANT.
- Technical Officer: Update and read access for attendance-related tables.
- Student: Read-only access to personal marks and attendance.

These access levels ensure confidentiality, integrity, and controlled data modification.

Database Accounts/Users and Reasons for Creation

Our database contains the following users and DB accounts.

- **Administrator (Admin)**
  - Admin has full access in database. Such as Create, delete, insert etc...
  - To manage and maintain the databases.

- **Dean**
  - Dean can access the all tables in database.
  - To Update tables and data.

- **Lecturer**
  - Lecturer can access table.
  - To manage database.

- **Technical officer**
  - To manage, maintain and optimize the database.
  - Read, write and update permissions for attendance related tables/views.

- **Student**
  - Access course tables and can only view marks.

Code Snippets to Support the Work

View result

```
drop view  IF EXISTS  Result;

CREATE VIEW Result AS SELECT mark_id,Mark.student_id,course_code,
 (( (quiz_1 + quiz_2 + quiz_3) - LEAST(quiz_1, quiz_2, quiz_3)) * 0.10)  AS marks_of_best_two_quizes,
 (assesment *0.05) AS Assesment_marks,(mid *0.20) AS mid_new,

 ( (( (quiz_1 + quiz_2 + quiz_3) - LEAST(quiz_1, quiz_2, quiz_3)) * 0.10) +  (assesment *0.05) + (mid *0.20)) AS Final_CA,
CASE

 WHEN
 ( (( (quiz_1 + quiz_2 + quiz_3) - LEAST(quiz_1, quiz_2, quiz_3)) * 0.10) +  (assesment *0.05) + (mid *0.20)) >=16  THEN 'EL'

 ELSE  'Not Eligible'

 END  AS  CA_Eligibility
 (end * 0.6) AS end_new,
  CASE
 when
   (end * 0.6) >= 21
 THEN 'ESA PASS'
 ELSE
  'ESA FAIL'  END AS END_Eligibility

 FROM Mark
 INNER JOIN Student ON Mark.student_id = Student.Reg_no
ORDER BY  Student.Reg_no  ASC ;
```

Output:

```
mysql> select * from Result;
```

mark_id	student_id	course_code	marks_of_best_two_quizes	Assesment_marks	mid_new	Final_CA	CA_Eligibility	end_new	END_Eligibility
M01	TG0001	ICT1233	11.00	3.95	13.40	28.35	EL	26.4	ESA PASS
M02	TG0001	TMS1233	13.90	4.25	14.80	32.95	EL	23.4	ESA PASS
M03	TG0001	ICT1253	13.60	4.00	13.20	30.80	EL	50.4	ESA PASS
M04	TG0001	ICT1242	11.40	4.35	12.80	28.55	EL	27.0	ESA PASS
M05	TG0001	ICT1212	14.20	4.20	14.00	32.40	EL	27.0	ESA PASS
M06	TG0001	ICT1222	13.50	4.05	8.00	25.55	EL	54.0	ESA PASS
M07	TG0001	TCS1212	14.30	3.90	14.60	32.80	EL	19.2	ESA FAIL
M08	TG0001	ENG1222	14.30	4.15	13.80	32.25	EL	24.6	ESA PASS
M09	TG0002	ICT1233	17.30	3.05	13.60	33.95	EL	42.0	ESA PASS
M10	TG0002	TMS1233	16.30	0.00	18.60	34.90	EL	30.0	ESA PASS
M11	TG0002	ICT1253	17.00	2.25	18.40	37.65	EL	36.0	ESA PASS
M12	TG0002	ICT1242	17.40	4.10	6.20	27.70	EL	57.6	ESA PASS
M13	TG0002	ICT1212	17.30	4.30	6.60	28.20	EL	54.6	ESA PASS
M14	TG0002	ICT1222	18.20	4.05	8.60	30.85	EL	39.6	ESA PASS
M15	TG0002	TCS1212	17.30	4.00	7.00	28.30	EL	54.0	ESA PASS
M16	TG0002	ENG1222	18.50	4.35	7.60	30.45	EL	58.8	ESA PASS
M17	TG0003	ICT1233	17.00	3.40	9.20	29.60	EL	54.0	ESA PASS
M18	TG0003	TMS1233	17.50	3.45	9.80	30.75	EL	21.6	ESA PASS
M19	TG0003	ICT1253	17.40	3.55	9.40	30.35	EL	56.4	ESA PASS
M20	TG0003	ICT1242	17.00	3.35	12.00	32.35	EL	22.2	ESA PASS
M21	TG0003	ICT1212	17.90	3.60	10.00	31.50	EL	23.4	ESA PASS
M22	TG0003	ICT1222	17.40	3.50	8.00	28.90	EL	29.4	ESA PASS
M23	TG0003	TCS1212	17.00	3.45	9.00	29.45	EL	21.0	ESA PASS
M24	TG0003	ENG1222	17.80	3.50	0.00	21.30	EL	35.4	ESA PASS
M25	TG0004	ICT1233	14.80	2.25	10.00	27.05	EL	40.8	ESA PASS
M26	TG0004	TMS1233	14.90	3.05	6.00	23.95	EL	36.0	ESA PASS
M27	TG0004	ICT1253	14.20	2.40	12.80	29.40	EL	42.0	ESA PASS
M28	TG0004	ICT1242	12.50	2.50	12.60	27.60	EL	38.4	ESA PASS
M29	TG0004	ICT1212	14.80	2.30	13.00	30.10	EL	39.0	ESA PASS
M30	TG0004	ICT1222	14.10	2.35	14.00	30.45	EL	39.6	ESA PASS
M31	TG0004	TCS1212	14.60	2.20	12.40	29.20	EL	36.0	ESA PASS
M32	TG0004	ENG1222	14.30	2.25	12.20	28.75	EL	37.2	ESA PASS
M33	TG0005	ICT1233	13.20	2.35	18.20	33.75	EL	48.0	ESA PASS
M34	TG0005	TMS1233	12.80	2.20	18.60	33.60	EL	47.4	ESA PASS

-- final marks ---

```
CREATE VIEW Final_Marks AS
SELECT
  m.student_id AS student_id,
  m.course_code AS course_code,
  CASE
    WHEN c.CA_Eligibility = 'EL' THEN
      CASE
        WHEN c.END_Eligibility = 'ESA PASS' THEN
          c.end_new + c.Final_CA
        ELSE 'ESA Fail'
      END
    ELSE 'CA Fail'
  END AS Final_Marks
FROM
  mark m
INNER JOIN
  Result c
  ON m.mark_id = c.mark_id

ORDER BY
  m.student_id ASC;
```

Output :

```
mysql> select * from Final_marks;
+------------+-------------+-------------+
| student_id | course_code | Final_Marks |
+------------+-------------+-------------+
| TG0001     | ICT1233     | 54.75       |
| TG0001     | TMS1233     | 56.35       |
| TG0001     | ICT1253     | 81.20       |
| TG0001     | ICT1242     | 55.55       |
| TG0001     | ICT1212     | 59.40       |
| TG0001     | ICT1222     | 79.55       |
| TG0001     | TCS1212     | ESA Fail    |
| TG0001     | ENG1222     | 56.85       |
| TG0002     | ICT1233     | 75.95       |
| TG0002     | TMS1233     | 64.90       |
| TG0002     | ICT1253     | 73.65       |
| TG0002     | ICT1242     | 85.30       |
| TG0002     | ICT1212     | 82.80       |
| TG0002     | ICT1222     | 70.45       |
| TG0002     | TCS1212     | 82.30       |
| TG0002     | ENG1222     | 89.25       |
| TG0003     | ICT1233     | 83.60       |
| TG0003     | TMS1233     | 52.35       |
| TG0003     | ICT1253     | 86.75       |
| TG0003     | ICT1242     | 54.55       |
| TG0003     | ICT1212     | 54.90       |
| TG0003     | ICT1222     | 58.30       |
| TG0003     | TCS1212     | 50.45       |
| TG0003     | ENG1222     | 56.70       |
| TG0004     | ICT1233     | 67.85       |
```

-- find grade –

```
CREATE VIEW Student_grade AS SELECT f. student_id,f.course_code,c.Credit,f.Final_Marks,

 CASE

    WHEN f.Final_Marks >= 85 AND f.Final_Marks <= 100 THEN 'A+'
    WHEN f.Final_Marks >= 75 AND f.Final_Marks < 85 THEN 'A'
    WHEN f.Final_Marks >= 70 AND f.Final_Marks < 75 THEN 'A-'
    WHEN f.Final_Marks >= 65 AND f.Final_Marks < 70 THEN 'B+'
    WHEN f.Final_Marks >= 60 AND f.Final_Marks < 65 THEN 'B'
    WHEN f.Final_Marks >= 55 AND f.Final_Marks < 60 THEN 'B-'
    WHEN f.Final_Marks >= 50 AND f.Final_Marks < 55 THEN 'C+'
    WHEN f.Final_Marks >= 45 AND f.Final_Marks < 50 THEN 'C'
    WHEN f.Final_Marks >= 40 AND f.Final_Marks < 45 THEN 'C-'
    WHEN f.Final_Marks >= 35 AND f.Final_Marks < 40 THEN 'D'
    WHEN f.Final_Marks >= 0 AND f.Final_Marks < 35 THEN 'E'
  END AS Grade,

  CASE

    WHEN f.Final_Marks >= 85 AND f.Final_Marks <= 100 THEN 4.0
    WHEN f.Final_Marks >= 75 AND f.Final_Marks < 85 THEN 4.0
    WHEN f.Final_Marks >= 70 AND f.Final_Marks < 75 THEN 3.7
    WHEN f.Final_Marks >= 65 AND f.Final_Marks < 70 THEN 3.3
    WHEN f.Final_Marks >= 60 AND f.Final_Marks < 65 THEN 3.0
    WHEN f.Final_Marks >= 55 AND f.Final_Marks < 60 THEN 2.7
    WHEN f.Final_Marks >= 50 AND f.Final_Marks < 55 THEN 2.3
    WHEN f.Final_Marks >= 45 AND f.Final_Marks < 50 THEN 2.0
    WHEN f.Final_Marks >= 40 AND f.Final_Marks < 45 THEN 1.7
    WHEN f.Final_Marks >= 35 AND f.Final_Marks < 40 THEN 1.3
    WHEN f.Final_Marks >= 0 AND f.Final_Marks < 35 THEN 0.0
  END AS Grade_Point

  from Final_Marks f


  INNER JOIN Course c ON c.C_code = f.course_code ;
```

Output:

```
mysql> select * from Student_grade;
+------------+-------------+--------+-------------+-------+-------------+
| student_id | course_code | Credit | Final_Marks | Grade | Grade_Point |
+------------+-------------+--------+-------------+-------+-------------+
| TG0001     | ENG1222     |      2 | 56.85       | B-    |         2.7 |
| TG0013     | ENG1222     |      2 | 47.85       | C     |         2.0 |
| TG0014     | ENG1222     |      2 | 88.55       | A+    |         4.0 |
| TG0015     | ENG1222     |      2 | 55.05       | B-    |         2.7 |
| TG0002     | ENG1222     |      2 | 89.25       | A+    |         4.0 |
| TG0003     | ENG1222     |      2 | 56.70       | B-    |         2.7 |
| TG0004     | ENG1222     |      2 | 65.95       | B+    |         3.3 |
| TG0005     | ENG1222     |      2 | 78.35       | A     |         4.0 |
| TG0006     | ENG1222     |      2 | CA Fail     | E     |         0.0 |
| TG0007     | ENG1222     |      2 | ESA Fail    | E     |         0.0 |
| TG0008     | ENG1222     |      2 | ESA Fail    | E     |         0.0 |
| TG0009     | ENG1222     |      2 | ESA Fail    | E     |         0.0 |
| TG0010     | ENG1222     |      2 | ESA Fail    | E     |         0.0 |
| TG0011     | ENG1222     |      2 | 71.85       | A-    |         3.7 |
| TG0012     | ENG1222     |      2 | 79.95       | A     |         4.0 |
| TG0001     | ICT1212     |      2 | 59.40       | B-    |         2.7 |
| TG0013     | ICT1212     |      2 | 78.30       | A     |         4.0 |
| TG0014     | ICT1212     |      2 | 87.65       | A+    |         4.0 |
| TG0015     | ICT1212     |      2 | 74.80       | A-    |         3.7 |
| TG0002     | ICT1212     |      2 | 82.80       | A     |         4.0 |
| TG0003     | ICT1212     |      2 | 54.90       | C+    |         2.3 |
| TG0004     | ICT1212     |      2 | 69.10       | B+    |         3.3 |
```

-- Grade Credit of students

```
CREATE VIEW Grade_Point_Credit  AS
SELECT  student_id,Credit,Course_code,(Grade_Point * Credit) AS pointCreditvalue FROM
Student_grade ;
```

Output:

```
mysql> select * from Grade_Point_Credit;
+------------+--------+-------------+------------------+
| student_id | Credit | Course_code | pointCreditvalue |
+------------+--------+-------------+------------------+
| TG0001     |      2 | ENG1222     |              5.4 |
| TG0013     |      2 | ENG1222     |              4.0 |
| TG0014     |      2 | ENG1222     |              8.0 |
| TG0015     |      2 | ENG1222     |              5.4 |
| TG0002     |      2 | ENG1222     |              8.0 |
| TG0003     |      2 | ENG1222     |              5.4 |
| TG0004     |      2 | ENG1222     |              6.6 |
| TG0005     |      2 | ENG1222     |              8.0 |
| TG0006     |      2 | ENG1222     |              0.0 |
| TG0007     |      2 | ENG1222     |              0.0 |
| TG0008     |      2 | ENG1222     |              0.0 |
| TG0009     |      2 | ENG1222     |              0.0 |
| TG0010     |      2 | ENG1222     |              0.0 |
| TG0011     |      2 | ENG1222     |              7.4 |
| TG0012     |      2 | ENG1222     |              8.0 |
| TG0001     |      2 | ICT1212     |              5.4 |
| TG0013     |      2 | ICT1212     |              8.0 |
| TG0014     |      2 | ICT1212     |              8.0 |
| TG0015     |      2 | ICT1212     |              7.4 |
| TG0002     |      2 | ICT1212     |              8.0 |
| TG0003     |      2 | ICT1212     |              4.6 |
| TG0004     |      2 | ICT1212     |              6.6 |
| TG0005     |      2 | ICT1212     |              8.0 |
| TG0006     |      2 | ICT1212     |              0.0 |
```

-- Cal SEM GPA

```
CREATE VIEW SGPA_check AS
SELECT g.student_id,(SUM(g.pointCreditvalue))/ SUM(g.Credit) AS SGPA
FROM Grade_Point_Credit g
INNER JOIN Course c  ON g.course_code = c.C_code
GROUP BY g.student_id
order by student_id;
```

Output:

```
mysql> select * from SGPA_check;
+------------+---------+
| student_id | SGPA    |
+------------+---------+
| TG0001     | 2.69474 |
| TG0002     | 3.76316 |
| TG0003     | 2.92105 |
| TG0004     | 3.31053 |
| TG0005     | 3.92105 |
| TG0006     | 0.76842 |
| TG0007     | 0.63158 |
| TG0008     | 0.96842 |
| TG0009     | 0.24211 |
| TG0010     | 1.96316 |
| TG0011     | 3.53158 |
| TG0012     | 0.73684 |
| TG0013     | 3.36842 |
| TG0014     | 1.54737 |
| TG0015     | 1.87368 |
+------------+---------+
```

-- CAL CURRENT GPA

```
drop view if exists CGPA_check;

CREATE VIEW  CGPA_check AS

SELECT g.student_id,(SUM(g.pointCreditvalue))/ SUM(g.Credit) AS CGPA

FROM Grade_Point_Credit g

INNER JOIN Course c ON g.course_code = c.C_code

 GROUP BY g.student_id

 order by student_id;
```

Output:

```
mysql> select * from CGPA_check;
+------------+---------+
| student_id | CGPA    |
+------------+---------+
| TG0001     | 2.69474 |
| TG0002     | 3.76316 |
| TG0003     | 2.92105 |
| TG0004     | 3.31053 |
| TG0005     | 3.92105 |
| TG0006     | 0.76842 |
| TG0007     | 0.63158 |
| TG0008     | 0.96842 |
| TG0009     | 0.24211 |
| TG0010     | 1.96316 |
| TG0011     | 3.53158 |
| TG0012     | 0.73684 |
| TG0013     | 3.36842 |
| TG0014     | 1.54737 |
| TG0015     | 1.87368 |
+------------+---------+
```

80% < attendance eligible or Not eligible (with medical)

```
CREATE VIEW Attendance_Eligibility_OR_NOT AS
  SELECT
   student_id,
   course_code,
     ROUND(COUNT(CASE WHEN att_state = 'Present' OR medical_id IS NOT NULL THEN 1 END) * 100.0 / 15,2) AS
Attendance_Percentage,
     IF(COUNT(CASE WHEN att_state = 'Present' OR medical_id IS NOT NULL THEN 1 END) * 100.0 / 15 >= 80, 'Eligible', 'Not Eligible')
AS Eligibility
  FROM
     Attendence
  GROUP BY
     student_id, course_code
   ORDER BY student_id ASC;
```

Output:

```
mysql> select * from Attendance_Eligibility_OR_NOT;
+------------+-------------+----------------------+--------------+
| student_id | course_code | Attendance_Percentage | Eligibility  |
+------------+-------------+----------------------+--------------+
| TG0001     | ENG1222     |               100.00 | Eligible     |
| TG0001     | ICT1212     |                93.33 | Eligible     |
| TG0001     | ICT1222     |                93.33 | Eligible     |
| TG0001     | ICT1233     |                93.33 | Eligible     |
| TG0001     | ICT1242     |                86.67 | Eligible     |
| TG0001     | ICT1253     |                80.00 | Eligible     |
| TG0001     | TCS1212     |               100.00 | Eligible     |
| TG0001     | TMS1233     |                86.67 | Eligible     |
| TG0002     | ENG1222     |                80.00 | Eligible     |
| TG0002     | ICT1212     |                93.33 | Eligible     |
| TG0002     | ICT1222     |                93.33 | Eligible     |
| TG0002     | ICT1233     |                80.00 | Eligible     |
| TG0002     | ICT1242     |               100.00 | Eligible     |
| TG0002     | ICT1253     |               100.00 | Eligible     |
| TG0002     | TCS1212     |                93.33 | Eligible     |
| TG0002     | TMS1233     |                86.67 | Eligible     |
| TG0003     | ENG1222     |                73.33 | Not Eligible |
| TG0003     | ICT1212     |                93.33 | Eligible     |
| TG0003     | ICT1222     |                93.33 | Eligible     |
| TG0003     | ICT1233     |                73.33 | Not Eligible |
| TG0003     | ICT1242     |                93.33 | Eligible     |
| TG0003     | ICT1253     |                80.00 | Eligible     |
| TG0003     | TCS1212     |               100.00 | Eligible     |
| TG0003     | TMS1233     |                93.33 | Eligible     |
| TG0004     | ENG1222     |                93.33 | Eligible     |
| TG0004     | ICT1212     |                80.00 | Eligible     |
| TG0004     | ICT1222     |                80.00 | Eligible     |
| TG0004     | ICT1233     |                66.67 | Not Eligible |
| TG0004     | ICT1242     |                93.33 | Eligible     |
```

-- Eligibility_with_attendence_and_CA

```
drop view if exists Eligibility_with_attendence_and_CA;
CREATE VIEW Eligibility_with_attendence_and_CA AS

select r.student_id as student_id ,r.course_code as course_code,r.CA_Eligibility AS
CA_Eligibility,a.Eligibility AS Att_Eligibility,


  IF(r.CA_Eligibility ='EL' AND a.Eligibility='Eligible','Eligible', 'Not Eligible') AS
Both_Attendence_and_CA_Eligibility

from Result r

INNER join Attendence_Eligibility_OR_NOT a ON r.student_id=a.student_id AND r.course_code=
a.course_code ;
```

Output:

```
mysql> select * from Eligibility_with_attendence_and_CA;
+------------+-------------+----------------+-----------------+-----------------------------------+
| student_id | course_code | CA_Eligibility | Att_Eligibility | Both_Attendence_and_CA_Eligibility |
+------------+-------------+----------------+-----------------+-----------------------------------+
| TG0001     | ENG1222     | EL             | Eligible        | Eligible                          |
| TG0001     | ICT1212     | EL             | Eligible        | Eligible                          |
| TG0001     | ICT1222     | EL             | Eligible        | Eligible                          |
| TG0001     | ICT1233     | EL             | Eligible        | Eligible                          |
| TG0001     | ICT1242     | EL             | Eligible        | Eligible                          |
| TG0001     | ICT1253     | EL             | Eligible        | Eligible                          |
| TG0001     | TCS1212     | EL             | Eligible        | Eligible                          |
| TG0001     | TMS1233     | EL             | Eligible        | Eligible                          |
| TG0002     | ENG1222     | EL             | Eligible        | Eligible                          |
| TG0002     | ICT1212     | EL             | Eligible        | Eligible                          |
| TG0002     | ICT1222     | EL             | Eligible        | Eligible                          |
| TG0002     | ICT1233     | EL             | Eligible        | Eligible                          |
| TG0002     | ICT1242     | EL             | Eligible        | Eligible                          |
| TG0002     | ICT1253     | EL             | Eligible        | Eligible                          |
| TG0002     | TCS1212     | EL             | Eligible        | Eligible                          |
| TG0002     | TMS1233     | EL             | Eligible        | Eligible                          |
| TG0003     | ENG1222     | EL             | Not Eligible    | Not Eligible                      |
| TG0003     | ICT1212     | EL             | Eligible        | Eligible                          |
| TG0003     | ICT1222     | EL             | Eligible        | Eligible                          |
| TG0003     | ICT1233     | EL             | Not Eligible    | Not Eligible                      |
| TG0003     | ICT1242     | EL             | Eligible        | Eligible                          |
| TG0003     | ICT1253     | EL             | Eligible        | Eligible                          |
| TG0003     | TCS1212     | EL             | Eligible        | Eligible                          |
| TG0003     | TMS1233     | EL             | Eligible        | Eligible                          |
| TG0004     | ENG1222     | EL             | Eligible        | Eligible                          |
| TG0004     | ICT1212     | EL             | Eligible        | Eligible                          |
| TG0004     | ICT1222     | EL             | Eligible        | Eligible                          |
| TG0004     | ICT1233     | EL             | Not Eligible    | Not Eligible                      |
| TG0004     | ICT1242     | EL             | Eligible        | Eligible                          |
| TG0004     | ICT1253     | EL             | Eligible        | Eligible                          |
| TG0004     | TCS1212     | EL             | Eligible        | Eligible                          |
| TG0004     | TMS1233     | EL             | Eligible        | Eligible                          |
| TG0005     | ENG1222     | EL             | Eligible        | Eligible                          |
| TG0005     | ICT1212     | EL             | Eligible        | Eligible                          |
| TG0005     | ICT1222     | EL             | Eligible        | Eligible                          |
| TG0005     | ICT1233     | EL             | Eligible        | Eligible                          |
| TG0005     | ICT1242     | EL             | Eligible        | Eligible                          |
| TG0005     | ICT1253     | EL             | Eligible        | Eligible                          |
| TG0005     | TCS1212     | EL             | Eligible        | Eligible                          |
| TG0005     | TMS1233     | EL             | Eligible        | Eligible                          |
| TG0006     | ENG1222     | Not Eligible   | Eligible        | Not Eligible                      |
| TG0006     | ICT1212     | EL             | Eligible        | Eligible                          |
| TG0006     | ICT1222     | EL             | Eligible        | Eligible                          |
| TG0006     | ICT1233     | Not Eligible   | Eligible        | Not Eligible                      |
| TG0006     | ICT1242     | EL             | Eligible        | Eligible                          |
| TG0006     | ICT1253     | EL             | Eligible        | Eligible                          |
```

Procedure

-- marks details whole batch

```
DELIMITER //

CREATE PROCEDURE marks_details()
BEGIN
SELECT mark_id,student_id,course_code, CA_Eligibility,END_Eligibility
FROM RESULT;
END //

DELIMITER ;
CALL marks_details();
```

Output :

```
mysql> CALL marks_details();
+---------+------------+-------------+----------------+-----------------+
| mark_id | student_id | course_code | CA_Eligibility | END_Eligibility |
+---------+------------+-------------+----------------+-----------------+
| M01     | TG0001     | ICT1233     | EL             | ESA PASS        |
| M02     | TG0001     | TMS1233     | EL             | ESA PASS        |
| M03     | TG0001     | ICT1253     | EL             | ESA PASS        |
| M04     | TG0001     | ICT1242     | EL             | ESA PASS        |
| M05     | TG0001     | ICT1212     | EL             | ESA PASS        |
| M06     | TG0001     | ICT1222     | EL             | ESA PASS        |
| M07     | TG0001     | TCS1212     | EL             | ESA FAIL        |
| M08     | TG0001     | ENG1222     | EL             | ESA PASS        |
| M09     | TG0002     | ICT1233     | EL             | ESA PASS        |
| M10     | TG0002     | TMS1233     | EL             | ESA PASS        |
| M11     | TG0002     | ICT1253     | EL             | ESA PASS        |
| M12     | TG0002     | ICT1242     | EL             | ESA PASS        |
| M13     | TG0002     | ICT1212     | EL             | ESA PASS        |
| M14     | TG0002     | ICT1222     | EL             | ESA PASS        |
| M15     | TG0002     | TCS1212     | EL             | ESA PASS        |
| M16     | TG0002     | ENG1222     | EL             | ESA PASS        |
| M17     | TG0003     | ICT1233     | EL             | ESA PASS        |
| M18     | TG0003     | TMS1233     | EL             | ESA PASS        |
| M19     | TG0003     | ICT1253     | EL             | ESA PASS        |
| M20     | TG0003     | ICT1242     | EL             | ESA PASS        |
```

-- MARKS DETAILS FOR EACH STUDENT (GIVEN STUDENT ID)

```
DELIMITER //

CREATE PROCEDURE marks_details_by_student_id( IN stu_num VARCHAR(10))
BEGIN
SELECT mark_id,student_id,course_code, CA_Eligibility,END_Eligibility
FROM RESULT
WHERE student_id=stu_num;
END //

DELIMITER ;
CALL marks_details_by_student_id('TG0001');
```

Output:

```
mysql> CALL marks_details_by_student_id('TG0001');
+---------+------------+-------------+----------------+-----------------+
| mark_id | student_id | course_code | CA_Eligibility | END_Eligibility |
+---------+------------+-------------+----------------+-----------------+
| M01     | TG0001     | ICT1233     | EL             | ESA PASS        |
| M02     | TG0001     | TMS1233     | EL             | ESA PASS        |
| M03     | TG0001     | ICT1253     | EL             | ESA PASS        |
| M04     | TG0001     | ICT1242     | EL             | ESA PASS        |
| M05     | TG0001     | ICT1212     | EL             | ESA PASS        |
| M06     | TG0001     | ICT1222     | EL             | ESA PASS        |
| M07     | TG0001     | TCS1212     | EL             | ESA FAIL        |
| M08     | TG0001     | ENG1222     | EL             | ESA PASS        |
+---------+------------+-------------+----------------+-----------------+
```

-- STUDENT GRADE FOR WHOLE BATCH

```
DROP PROCEDURE IF EXISTS marks_grade_details;
DELIMITER //

CREATE PROCEDURE marks_grade_details()
BEGIN


SELECT student_id,course_code, Final_Marks,Grade,Grade_Point


FROM Student_grade;

END //


DELIMITER ;


CALL marks_grade_details();
```

Output:

```
mysql> CALL marks_grade_details();
+------------+-------------+-------------+-------+-------------+
| student_id | course_code | Final_Marks | Grade | Grade_Point |
+------------+-------------+-------------+-------+-------------+
| TG0001     | ICT1212     | 59.40       | B-    |         2.7 |
| TG0013     | ICT1212     | 78.30       | A     |         4.0 |
| TG0014     | ICT1212     | 87.65       | A+    |         4.0 |
| TG0015     | ICT1212     | 74.80       | A-    |         3.7 |
| TG0002     | ICT1212     | 82.80       | A     |         4.0 |
| TG0003     | ICT1212     | 54.90       | C+    |         2.3 |
| TG0004     | ICT1212     | 69.10       | B+    |         3.3 |
| TG0005     | ICT1212     | 81.70       | A     |         4.0 |
| TG0006     | ICT1212     | ESA Fail    | E     |         0.0 |
| TG0007     | ICT1212     | CA Fail     | E     |         0.0 |
| TG0008     | ICT1212     | ESA Fail    | E     |         0.0 |
| TG0009     | ICT1212     | ESA Fail    | E     |         0.0 |
| TG0010     | ICT1212     | ESA Fail    | E     |         0.0 |
| TG0011     | ICT1212     | 58.70       | B-    |         2.7 |
| TG0012     | ICT1212     | 60.80       | B     |         3.0 |
| TG0001     | ICT1222     | 79.55       | A     |         4.0 |
| TG0013     | ICT1222     | 91.20       | A+    |         4.0 |
| TG0014     | ICT1222     | 57.60       | B-    |         2.7 |
| TG0015     | ICT1222     | CA Fail     | E     |         0.0 |
| TG0002     | ICT1222     | 70.45       | A-    |         3.7 |
| TG0003     | ICT1222     | 58.30       | B-    |         2.7 |
| TG0004     | ICT1222     | 70.05       | A-    |         3.7 |
| TG0005     | ICT1222     | 84.45       | A     |         4.0 |
| TG0006     | ICT1222     | 45.10       | C     |         2.0 |
| TG0007     | ICT1222     | CA Fail     | E     |         0.0 |
| TG0008     | ICT1222     | 53.20       | C+    |         2.3 |
| TG0009     | ICT1222     | ESA Fail    | E     |         0.0 |
| TG0010     | ICT1222     | 72.45       | A-    |         3.7 |
| TG0011     | ICT1222     | 70.00       | A-    |         3.7 |
| TG0012     | ICT1222     | ESA Fail    | E     |         0.0 |
| TG0001     | ICT1233     | 54.75       | C+    |         2.3 |
| TG0002     | ICT1233     | 75.95       | A     |         4.0 |
| TG0014     | ICT1233     | ESA Fail    | E     |         0.0 |
| TG0015     | ICT1233     | 64.45       | B     |         3.0 |
| TG0003     | ICT1233     | 83.60       | A     |         4.0 |
| TG0004     | ICT1233     | 67.85       | B+    |         3.3 |
| TG0005     | ICT1233     | 81.75       | A     |         4.0 |
| TG0006     | ICT1233     | CA Fail     | E     |         0.0 |
```

-- STUDENT GRADE FOR GIVEN STUDENT ID

```
DROP PROCEDURE IF EXISTS marks_grade_details_by_student_id;
DELIMITER //

CREATE PROCEDURE marks_grade_details_by_student_id(IN stu_num VARCHAR(10))
BEGIN

SELECT student_id,course_code, Final_Marks,Grade,Grade_Point

FROM Student_grade
WHERE student_id=stu_num;
END //
DELIMITER ;
CALL  marks_grade_details_by_student_id ('TG0015');
```

Output :

```
mysql> CALL  marks_grade_details_by_student_id ('TG0015');
+------------+-------------+-------------+-------+-------------+
| student_id | course_code | Final_Marks | Grade | Grade_Point |
+------------+-------------+-------------+-------+-------------+
| TG0015     | ICT1233     | 64.45       | B     |         3.0 |
| TG0015     | TMS1233     | 54.90       | C+    |         2.3 |
| TG0015     | ICT1253     | 50.65       | C+    |         2.3 |
| TG0015     | ICT1242     | ESA Fail    | E     |         0.0 |
| TG0015     | ICT1212     | 74.80       | A-    |         3.7 |
| TG0015     | ICT1222     | CA Fail     | E     |         0.0 |
| TG0015     | TCS1212     | ESA Fail    | E     |         0.0 |
| TG0015     | ENG1222     | 55.05       | B-    |         2.7 |
+------------+-------------+-------------+-------+-------------+
8 rows in set (0.00 sec)
```

-- SEMESTER GPA DETAILS WHOLE BATCH

```
DROP PROCEDURE IF EXISTS sem_gpa_details;
DELIMITER //
CREATE PROCEDURE sem_gpa_details ()
BEGIN
 SELECT student_id,SGPA
 FROM SGPA_check;
END //
DELIMITER ;

CALL sem_gpa_details();
```

Output:

```
mysql> CALL sem_gpa_details();
+------------+---------+
| student_id | SGPA    |
+------------+---------+
| TG0001     | 2.69474 |
| TG0002     | 3.76316 |
| TG0003     | 2.92105 |
| TG0004     | 3.31053 |
| TG0005     | 3.92105 |
| TG0006     | 0.76842 |
| TG0007     | 0.63158 |
| TG0008     | 0.96842 |
| TG0009     | 0.24211 |
| TG0010     | 1.96316 |
| TG0011     | 3.53158 |
| TG0012     | 0.73684 |
| TG0013     | 3.36842 |
| TG0014     | 1.54737 |
| TG0015     | 1.87368 |
+------------+---------+
15 rows in set (0.00 sec)

Query OK, 0 rows affected, 504 warnings (0.01 sec)
```

-- SEMESTER GPA DETAILS FOR GIVEN STUDENT ID

```
DROP PROCEDURE IF EXISTS sem_gpa_details_details_by_student_id;
DELIMITER //
CREATE PROCEDURE sem_gpa_details_details_by_student_id (IN stu_num VARCHAR(10))
BEGIN
 SELECT student_id,SGPA
 FROM SGPA_check
 WHERE student_id=stu_num;
END //
DELIMITER ;

CALL sem_gpa_details_details_by_student_id('TG0004');
```

Output :

```
mysql> CALL sem_gpa_details_details_by_student_id('TG0004');
+------------+---------+
| student_id | SGPA    |
+------------+---------+
| TG0004     | 3.31053 |
+------------+---------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

-- CURRENT GPA DETAILS WHOLE BATCH

```
DROP PROCEDURE IF EXISTS cur_gpa_details;

DELIMITER //
CREATE PROCEDURE cur_gpa_details ()
BEGIN
 SELECT student_id,CGPA
 FROM CGPA_check;
END //

DELIMITER ;

CALL cur_gpa_details();
```

Output:

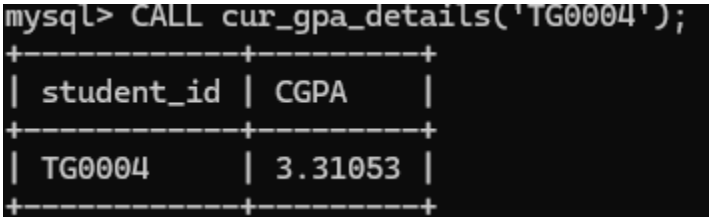
```
mysql> CALL cur_gpa_details();
+------------+---------+
| student_id | CGPA    |
+------------+---------+
| TG0001     | 2.69474 |
| TG0002     | 3.76316 |
| TG0003     | 2.92105 |
| TG0004     | 3.31053 |
| TG0005     | 3.92105 |
| TG0006     | 0.76842 |
| TG0007     | 0.63158 |
| TG0008     | 0.96842 |
| TG0009     | 0.24211 |
| TG0010     | 1.96316 |
| TG0011     | 3.53158 |
| TG0012     | 0.73684 |
| TG0013     | 3.36842 |
| TG0014     | 1.54737 |
| TG0015     | 1.87368 |
+------------+---------+
15 rows in set (0.00 sec)
```

-- CURRENT GPA DETAILS FOR GIVEN STUDENT ID

```
DROP PROCEDURE IF EXISTS cur_gpa_details_by_student_id;
DELIMITER //
CREATE PROCEDURE cur_gpa_details_by_student_id (IN stu_num VARCHAR(10))

BEGIN
 SELECT student_id,CGPA
 FROM CGPA_check
 WHERE student_id=stu_num;
END //
DELIMITER ;
CALL cur_gpa_details_by_student_id('TG0004');
```

Output :

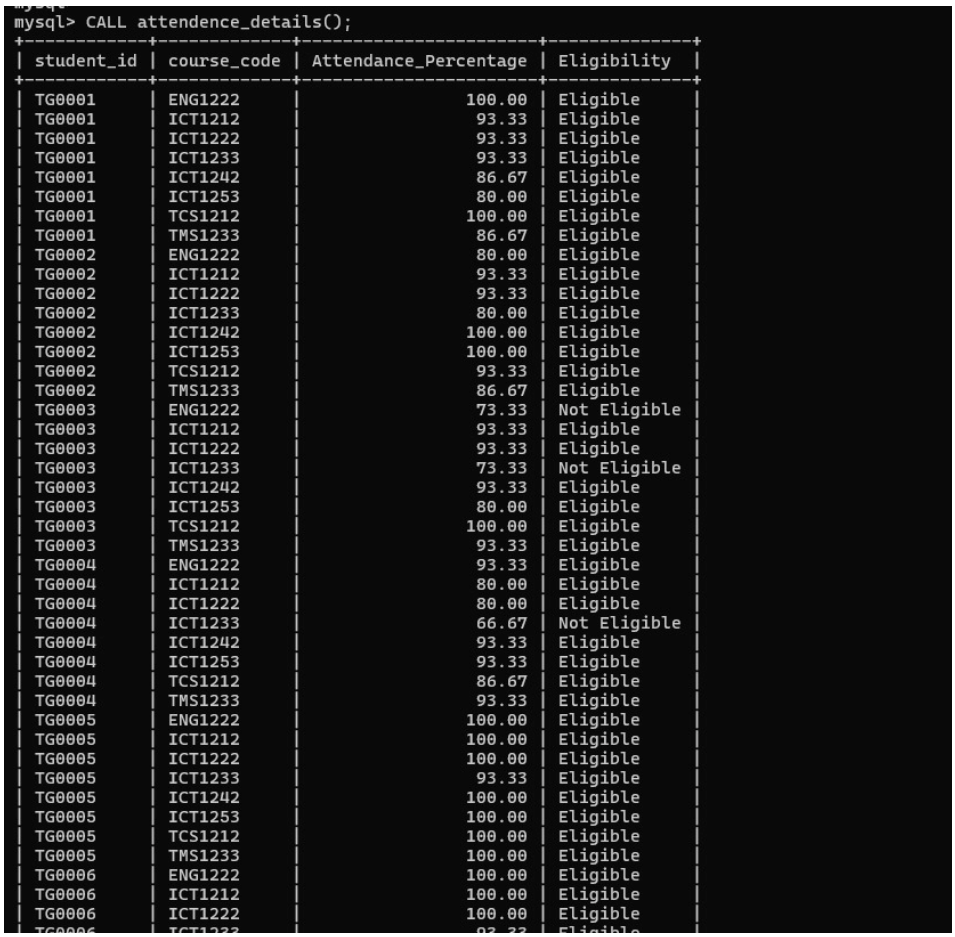


-- attendence DETAILS WHOLE BATCH

```
DROP PROCEDURE IF EXISTS attendence_details;
DELIMITER //
CREATE PROCEDURE attendence_details ()
BEGIN
 SELECT student_id,course_code,Attendance_Percentage,Eligibility
 FROM Attendence_Eligibility_OR_NOT;
END //
DELIMITER ;

CALL attendence_details();
```

Output:



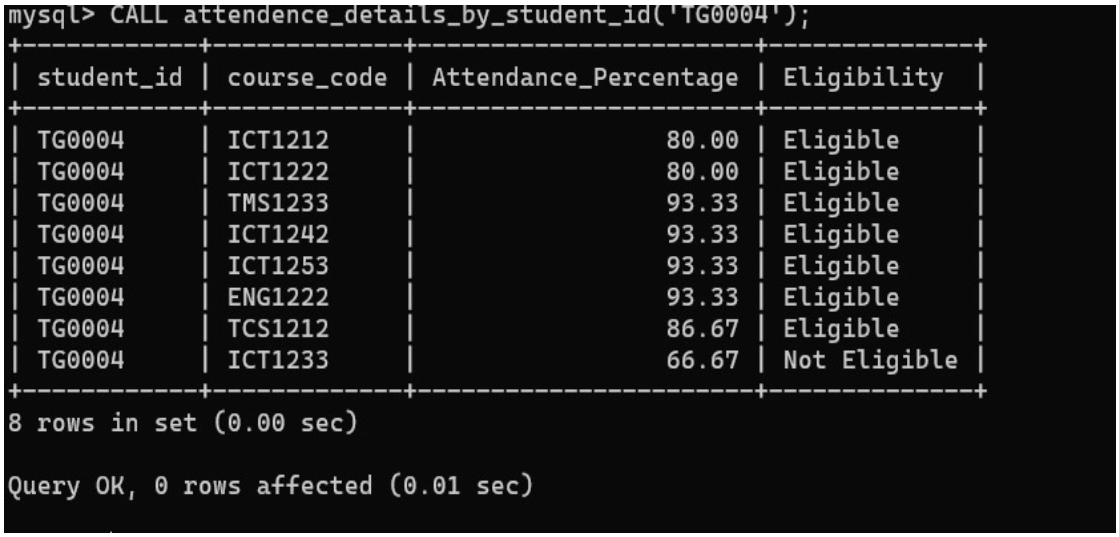
```
mysql> CALL attendence_details();
+------------+-------------+----------------------+--------------+
| student_id | course_code | Attendance_Percentage | Eligibility  |
+------------+-------------+----------------------+--------------+
| TG0001     | ENG1222     |               100.00 | Eligible     |
| TG0001     | ICT1212     |                93.33 | Eligible     |
| TG0001     | ICT1222     |                93.33 | Eligible     |
| TG0001     | ICT1233     |                93.33 | Eligible     |
| TG0001     | ICT1242     |                86.67 | Eligible     |
| TG0001     | ICT1253     |                80.00 | Eligible     |
| TG0001     | TCS1212     |               100.00 | Eligible     |
| TG0001     | TMS1233     |                86.67 | Eligible     |
| TG0002     | ENG1222     |                80.00 | Eligible     |
| TG0002     | ICT1212     |                93.33 | Eligible     |
| TG0002     | ICT1222     |                93.33 | Eligible     |
| TG0002     | ICT1233     |                80.00 | Eligible     |
| TG0002     | ICT1242     |               100.00 | Eligible     |
| TG0002     | ICT1253     |               100.00 | Eligible     |
| TG0002     | TCS1212     |                93.33 | Eligible     |
| TG0002     | TMS1233     |                86.67 | Eligible     |
| TG0003     | ENG1222     |                73.33 | Not Eligible |
| TG0003     | ICT1212     |                93.33 | Eligible     |
| TG0003     | ICT1222     |                93.33 | Eligible     |
| TG0003     | ICT1233     |                73.33 | Not Eligible |
| TG0003     | ICT1242     |                93.33 | Eligible     |
| TG0003     | ICT1253     |                80.00 | Eligible     |
| TG0003     | TCS1212     |               100.00 | Eligible     |
| TG0003     | TMS1233     |                93.33 | Eligible     |
| TG0004     | ENG1222     |                93.33 | Eligible     |
| TG0004     | ICT1212     |                80.00 | Eligible     |
| TG0004     | ICT1222     |                80.00 | Eligible     |
| TG0004     | ICT1233     |                66.67 | Not Eligible |
| TG0004     | ICT1242     |                93.33 | Eligible     |
| TG0004     | ICT1253     |                93.33 | Eligible     |
| TG0004     | TCS1212     |                86.67 | Eligible     |
| TG0004     | TMS1233     |                93.33 | Eligible     |
| TG0005     | ENG1222     |               100.00 | Eligible     |
| TG0005     | ICT1212     |               100.00 | Eligible     |
| TG0005     | ICT1222     |               100.00 | Eligible     |
| TG0005     | ICT1233     |                93.33 | Eligible     |
| TG0005     | ICT1242     |               100.00 | Eligible     |
| TG0005     | ICT1253     |               100.00 | Eligible     |
| TG0005     | TCS1212     |               100.00 | Eligible     |
| TG0005     | TMS1233     |               100.00 | Eligible     |
| TG0006     | ENG1222     |               100.00 | Eligible     |
| TG0006     | ICT1212     |               100.00 | Eligible     |
| TG0006     | ICT1222     |               100.00 | Eligible     |
```

-- attendance DETAILS FOR GIVEN STUDENT ID

```
DROP PROCEDURE IF EXISTS attendence_details__by_student_id;
DELIMITER //
CREATE PROCEDURE attendence_details_by_student_id (IN stu_num VARCHAR(10))
BEGIN
 SELECT student_id,course_code,Attendance_Percentage,Eligibility
 FROM Attendence_Eligibility_OR_NOT
 WHERE student_id=stu_num;
END //
DELIMITER ;
```

```
CALL attendence_details_by_student_id('TG0004');
```

Output :



```
mysql> CALL attendence_details_by_student_id('TG0004');
+------------+-------------+-----------------------+--------------+
| student_id | course_code | Attendance_Percentage | Eligibility  |
+------------+-------------+-----------------------+--------------+
| TG0004     | ICT1212     |                 80.00 | Eligible     |
| TG0004     | ICT1222     |                 80.00 | Eligible     |
| TG0004     | TMS1233     |                 93.33 | Eligible     |
| TG0004     | ICT1242     |                 93.33 | Eligible     |
| TG0004     | ICT1253     |                 93.33 | Eligible     |
| TG0004     | ENG1222     |                 93.33 | Eligible     |
| TG0004     | TCS1212     |                 86.67 | Eligible     |
| TG0004     | ICT1233     |                 66.67 | Not Eligible |
+------------+-------------+-----------------------+--------------+
8 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

-- Eligibility_with_attendence_and_CA for WHOLE BATCH

```
DROP PROCEDURE IF EXISTS Eligibility_with_attendence_and_CA_details;
DELIMITER //
CREATE PROCEDURE Eligibility_with_attendence_and_CA_details ()
BEGIN
 SELECT student_id,course_code,Both_Attendence_and_CA_Eligibility
 FROM Eligibility_with_attendence_and_CA;
END //
DELIMITER ;


CALL Eligibility_with_attendence_and_CA_details();
```

Output:

```
mysql>
mysql> CALL Eligibility_with_attendence_and_CA_details();
+------------+-------------+-----------------------------------+
| student_id | course_code | Both_Attendence_and_CA_Eligibility |
+------------+-------------+-----------------------------------+
| TG0001     | ENG1222     | Eligible                          |
| TG0001     | ICT1212     | Eligible                          |
| TG0001     | ICT1222     | Eligible                          |
| TG0001     | ICT1233     | Eligible                          |
| TG0001     | ICT1242     | Eligible                          |
| TG0001     | ICT1253     | Eligible                          |
| TG0001     | TCS1212     | Eligible                          |
| TG0001     | TMS1233     | Eligible                          |
| TG0002     | ENG1222     | Eligible                          |
| TG0002     | ICT1212     | Eligible                          |
| TG0002     | ICT1222     | Eligible                          |
| TG0002     | ICT1233     | Eligible                          |
| TG0002     | ICT1242     | Eligible                          |
| TG0002     | ICT1253     | Eligible                          |
| TG0002     | TCS1212     | Eligible                          |
| TG0002     | TMS1233     | Eligible                          |
| TG0003     | ENG1222     | Not Eligible                      |
| TG0003     | ICT1212     | Eligible                          |
| TG0003     | ICT1222     | Eligible                          |
| TG0003     | ICT1233     | Not Eligible                      |
| TG0003     | ICT1242     | Eligible                          |
| TG0003     | ICT1253     | Eligible                          |
| TG0003     | TCS1212     | Eligible                          |
| TG0003     | TMS1233     | Eligible                          |
| TG0004     | ENG1222     | Eligible                          |
| TG0004     | ICT1212     | Eligible                          |
| TG0004     | ICT1222     | Eligible                          |
| TG0004     | ICT1233     | Not Eligible                      |
| TG0004     | ICT1242     | Eligible                          |
| TG0004     | ICT1253     | Eligible                          |
| TG0004     | TCS1212     | Eligible                          |
| TG0004     | TMS1233     | Eligible                          |
```

-- Eligibility_with_attendence_and_CA for GIVEN STUDENT ID

```
DROP PROCEDURE IF EXISTS Eligibility_with_attendence_and_CA_details_by_student_id;
DELIMITER //
CREATE PROCEDURE Eligibility_with_attendence_and_CA_details_by_student_id (IN stu_num
VARCHAR(10))
BEGIN
 SELECT student_id,course_code,Both_Attendence_and_CA_Eligibility
 FROM Eligibility_with_attendence_and_CA
 WHERE student_id=stu_num;
END //
DELIMITER ;
```

```
CALL Eligibility_with_attendence_and_CA_details_by_student_id('TG0004');
```

Output:

```
mysql> CALL Eligibility_with_attendence_and_CA_details_by_student_id('TG0004');
+------------+-------------+-----------------------------------+
| student_id | course_code | Both_Attendence_and_CA_Eligibility |
+------------+-------------+-----------------------------------+
| TG0004     | ENG1222     | Eligible                          |
| TG0004     | ICT1212     | Eligible                          |
| TG0004     | ICT1222     | Eligible                          |
| TG0004     | ICT1233     | Not Eligible                      |
| TG0004     | ICT1242     | Eligible                          |
| TG0004     | ICT1253     | Eligible                          |
| TG0004     | TCS1212     | Eligible                          |
| TG0004     | TMS1233     | Eligible                          |
+------------+-------------+-----------------------------------+
8 rows in set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)
```

Problems Faced During Development

- Complex foreign key dependencies during table creation.
- Data inconsistency during testing of attendance records.
- Difficulty designing a normalized relational schema.
- Redundant data during initial inserts.
-Difficult to draw ER-diagram and relational schema using Draw.io

Solutions and How Problems Were Overcome

- Created tables in dependency order to resolve foreign key conflicts.
- Validated data sets before bulk insertion.
- Used online ER modeling tools and MySQL tutorials for relational design.
- Normalized tables to eliminate redundancy and ensure consistency.

Hosting the Backend

If hosted online, the database could be deployed on MySQL Cloud, AWS RDS, or Google Cloud SQL. These platforms offer scalability, automated backups, and global access. Hosting allows real-time data management by faculty members and students.

Cloud Hosting Changes

For cloud deployment, the following configurations are required:
- Use environment variables for credentials.
- Enable SSL for secure connections.
- Modify IP-based access control.
- Optimize indexes for large-scale query performance.

References:

- w3cschool.com
- Lecture Materials.
- YOUTUBE videos.

Individual Contribution to Backend Development

TG/2023/1711

B.P Kothalawala

I designed and implemented several essential tables, views, and stored procedures that together

1. CREATE DATABASE

- Firstly create a database as FacultyDB for work in mysql.

2. Create Database tables

I contributed to designing several essential tables that support academic and administrative

- *Department table and insert data* -

That department table includes ‘department id’ and ‘department_names’ to store details about departments in university.

- *Create student table and insert data –*

  That table was used to store details of all students in university. Student registration number is the primary key of this table, and I added a foreign key as department id to find student department. I used on delete and on update cascade when creating this table.

- *Create Lecture table and insert data –*

  That table is used for store lectures in university. Every lecture has a unique lecture id.

- *Create Course table and insert data -*

  This table use for getting details about what course enrolled in our university and which department.

3. Update mark table structure and add new values –

While doing my part, I had to make some changes to the marks table.

Delete some data and add new columns to this table.

4. Create views

- Create views for check result –

  That view calculate final CA marks , including (quizes, assesment, mid_marks) and CA Eligibity or Not acroding to university startegies of 2024.

- Create views for check final Marks -

  This view show final marks of students.some student can't see mark because they are not pass CA or End exam . Previous result table used to cal this table marks.

5. Create stored procedures

- I create two stored procedures
  to get summary mark of whole batch,
  to get summary mark of given student registration number.

TG/2023/1700

A.L. B. Abilasha.

Introduction

This report highlights my individual contribution to the development of the Learning Management System (LMS) database project. My work primarily focused on **designing and implementing relational database tables, creating SQL views and stored procedures, managing user accounts with privileges, and ensuring data integrity using cascading relationships.**

Database Table Design and Implementation

I was responsible for designing and creating several key tables for FacultyDB database. These include the **student–Course Relationship**, **Attendance**, **Admin**, and **medical tables**. Each table was structured with appropriate **primary and foreign keys** to maintain referential integrity. To automate dependency handling, I implemented **ON DELETE** and **ON UPDATE CASCADE** constraints across all relationships. This ensures that any changes in parent tables automatically reflect in child tables, preventing orphaned records and data inconsistency.

After defining the table structures, I entered the tables with realistic sample data to support system testing and demonstrations.

SQL Views for Grade and Point Calculation

I developed SQL views to automate the calculation and retrieval of **student grades (student_grade)** and **grade point (valuesGrade_Point_Credit)**. These views combine data from multiple tables such as **Marks**, **Courses**, and **Assessments** to generate consolidated academic summaries.

Stored Procedures for Academic Reports

To streamline repetitive academic tasks, I implemented stored procedures that automate grade calculation and report generation. The key procedures include:

- Procedure to calculate grades for all students within a batch. **(marks_grade_details ())**
- Procedure to calculate and display grades for a specific student using their Student ID. **(marks_grade_details ())**

These procedures minimized manual work, reduced query errors, and ensured consistency in academic evaluations.

User Management and Privilege Control

I created and managed user accounts with appropriate privileges to ensure database security and controlled access. User roles such as **Admin**, **Lecturer**, **Technical Officer**, and **Student** were defined with specific permissions. For instance, Admins were given full privileges with GRANT options, while Students had read-only access to grades and attendance records. This role-based access control safeguarded the system against unauthorized modifications.

Testing and Troubleshooting

I performed multiple test runs to validate table relationships, data insertion operations, and stored procedure logic. During testing, I resolved issues related to foreign key conflicts, cascading constraints, and syntax errors in SQL scripts. The debugging process enhanced system reliability and ensured that all components functioned smoothly.

Conclusion

Overall, my contribution focused on the design and development of critical backend functionalities that ensure the Learning Management System operates efficiently and securely. By creating key database tables, implementing CASCADE relationships, automating calculations through views and procedures, and enforcing user privileges, I contributed to building a robust, scalable, and maintainable academic database system.

TG/2023/1716

D. G. Iduwara.

Individual Contribution to the Backend Development

As a member of the Database Management System (DBMS) group project, my primary contribution focused on the development, optimization, and data management aspects of our Learning Management System (LMS) backend. My work emphasized ensuring database consistency, data accuracy, and supporting academic analytics through the creation of tables, data manipulation scripts, views, and stored procedures

Table Creation and Data Management

I was responsible for designing and implementing several core database tables that serve as the foundation for administrative and academic data management. These include:

- Dean, User, and Marks tables – defined with primary and foreign key constraints to ensure data integrity across all user and assessment records.
- Populated Dean, User, and Marks tables with initial data entries to support system testing and integration.
- Student–Course Relationship Table – maintained student enrollment records and implemented updates to synchronize course allocations and grade tracking.

SQL Views for GPA and Academic Tracking

To enhance the analytical capabilities of the system, I created multiple SQL Views that allow academic staff to efficiently track and evaluate student performance:

- Semester GPA View: Calculates semester-based GPA for each student by aggregating grade points and credits earned within a given semester.
- Current GPA View: Provides an overall cumulative GPA for each student, considering all completed courses.

Stored Procedures for GPA and Batch Reports

To streamline reporting and simplify repetitive queries, I implemented several stored procedures to handle both individual and batch-level academic reports:

- SemesterGPA_Details_Batch: Generates GPA reports for an entire batch within a given semester.
- SemesterGPA_ByStudent: Retrieves GPA details for a specific student based on their ID.
- CurrentGPA_Batch: Produces a cumulative GPA summary for all students in a batch.
- CurrentGPA_ByStudent: Returns the current cumulative GPA for an individual student.

Data Maintenance and Updates

I continuously maintained and updated data across multiple relational tables to ensure smooth system operation. This included correcting in student–course assignments, updating lecturer and department records during schema changes, and testing updates to maintain referential integrity after migrations or revisions.

TG/2023/1743

S.K.A.C.J.Samarasinghe

Introduction

- My main contribution to the Learning Management System (LMS) database project focused on the Attendance Management and Eligibility Evaluation modules. I worked mainly on creating tables, procedures, and views that automate attendance tracking, eligibility checking, and data maintenance. The goal was to make the system accurate, automated, and easy to manage for both students and staff.

Table Creation and Management

- Technical Officer Table: I designed and implemented the Technical Officer table to store officer details such as ID, name and email. This table helps assign responsibilities and maintain accountability for attendance management.

- Attendance Table Updates: I modified and updated the Attendance table structure to include additional attributes such as session type, attendance state and hour. I also ensured that all foreign key constraints were properly defined to maintain referential integrity.

Data Handling

- Entered sample data into the Technical Officer table for testing and demonstration purposes.
- Updated attendance records and maintained data accuracy through validation and constraints.
- Implemented queries to update and delete attendance data when changes occurred (such as approved medical leave).

Stored Procedures

I created several stored procedures to automate and simplify attendance-related processes:

- Get Attendance Details By Student ID – Retrieves all attendance details for a specific student.
- Get Attendance Details By Batch – Displays attendance summaries for a whole batch.
- Get Eligibility With Attendance And CA – Calculates eligibility status for students by combining attendance and continuous assessment (CA) marks.
- Get Eligibility For Batch – Determines exam eligibility for an entire batch of students based on institutional policies.

Views

To support faster data analysis and improve reporting:

- Attendance Eligibility View – Displays whether a student meets the required attendance percentage.
- Eligibility With Attendance and CA View – Combines attendance percentage and CA marks to determine overall eligibility.

These views help lecturers and administrators easily monitor student readiness for examinations.

Summary of Contribution

- My work focused on the attendance and eligibility modules of the LMS database. Through table design, data management, stored procedures, and analytical views, I ensured that the system automatically evaluates attendance and eligibility with accuracy and efficiency.